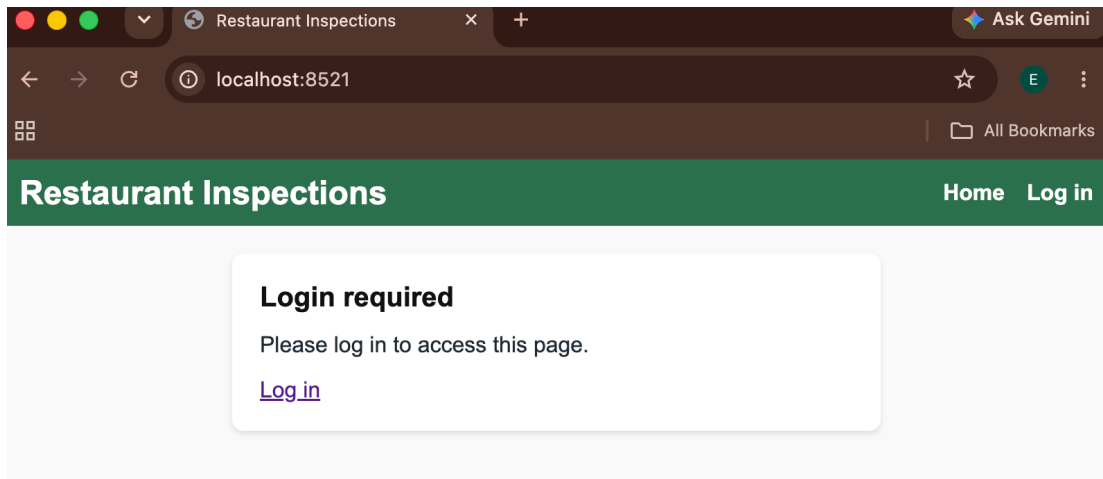


SID4: 1421
PORT_BASE: 8521
PREFIX: s1421
SEED: 1421
VERIFY_SEED: 2614221
DOMAIN_ID: 5
Hardware: Apple M2 Pro, 16GB
Local model used: qwen3:8b
Git link: <https://github.com/ericezhao/data260-1421>
Tag link: <https://github.com/ericezhao/data260-1421/releases/tag/hw4>
Tagged commit hash: 7bc6f448d197a89b4b82fb7d662d1ddd6bf3e42c

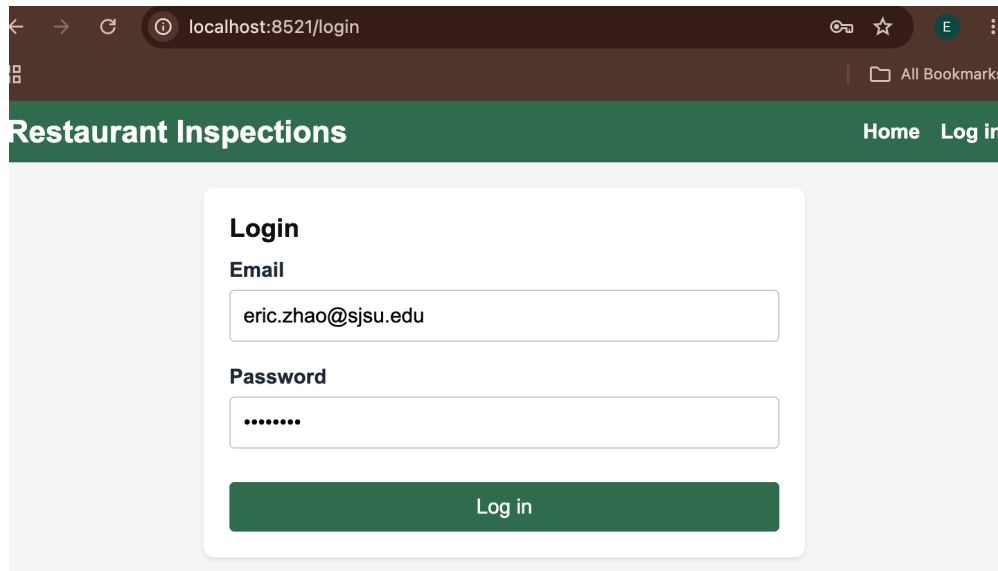
Part 1 – React Client

User not logged in:



```
18 function RequireAuth({ auth, children }) {
19   if (!auth.loggedIn) {
20     return (
21       <div className="card">
22         <h3>Login required</h3>
23         <p>Please log in to access this page.</p>
24         <Link to="/login">Log in</Link>
25       </div>
26     );
27   }
28   return children;
29 }
115 <nav className="heading-links">
116   <Link to="/">Home</Link>
117   {auth.loggedIn ? <Link to="/create">Add record</Link> : <Link to="/login">Log in</Link>}
118 </nav>
```

Login page:



The screenshot shows a web browser at the address `localhost:8521/login`. The page has a dark green header with the text "Restaurant Inspections" on the left and "Home" and "Log in" links on the right. Below the header is a light gray background with a white login card. The card has a title "Login" and two input fields: "Email" with the value "eric.zhao@sjsu.edu" and "Password" with masked characters ".....". A green "Log in" button is at the bottom of the card.

```
export default function Login({ auth, onLogin, onLogout }) {  
  // state variables to store the email and password  
  const [email, setEmail] = useState("");  
  const [password, setPassword] = useState("");  
  
  async function handleSubmit(event) {  
    // prevent the default behavior of the form - refreshing the page  
    event.preventDefault();  
    const success = await onLogin(email, password);  
    if (success) {  
      setEmail("");  
      setPassword("");  
    }  
  }  
  
  if (auth.loggedIn) {  
    return (  
      <div className="card">  
        <h3>Login</h3>  
        <p>You are logged in.</p>  
        <button className="secondary" onClick={onLogout}>  
          Log out  
        </button>  
      </div>  
    );  
  }  
  
  return (  

```

```

<div className="card">
  <h3>Login</h3>
  <form onSubmit={handleSubmit}>
    <label>
      Email
      <input type="email" value={email} onChange={(e) => setEmail(e.target.value)}
required />
    </label>
    <label>
      Password
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        required
      />
    </label>
    <button type="submit">Log in</button>
  </form>
</div>
);
}

```

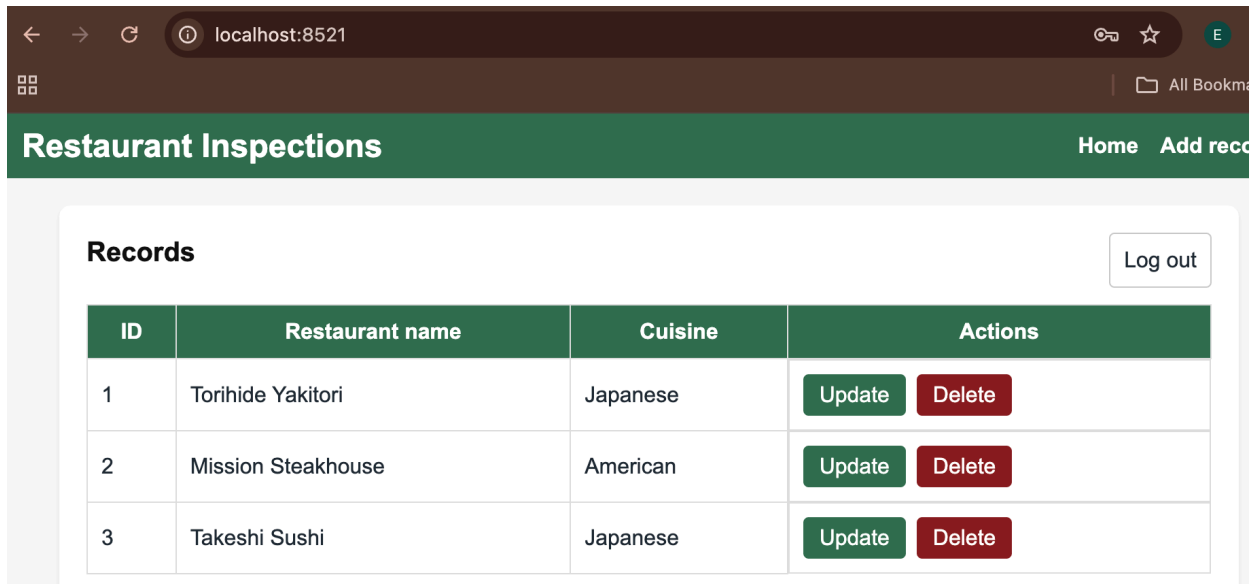
Logged in with cookie having no user data:

_xsrf	2f1f5e84a3 eaf74303877108e1c45879...	localhost	/	2026-10-09T02:...	59			
session_id	7f448a694fb27f7c08810dcb9cb3f5a3...	localhost	/	2026-09-27T01:1...	74	✓		L
username: localhost:8888	"215:01017388023625123:username:loc...	localhost	/	2026-10-09T02:...	265	✓		

```

@app.post("/auth/login")
def login(payload: schema.LoginIn, response: Response, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.email == payload.email).first()
    if not user or not verify_password(payload.password, user.password_hash):
        raise HTTPException(status_code=401, detail="Invalid email or password")
    session_row = create_session(db, user_id=user.id)
    response.set_cookie(
        key="session_id",
        value=session_row.id,
        httponly=True,
        samesite="lax",
        max_age=30 * 60,
    )
    return {"message": "logged in", "userId": user.id}

```

Home page:

```
export default function Home({ records, loading, onLogout }) {
  const navigate = useNavigate();

  return (
    <div className="card">
      <div className="card-top">
        <h3>Records</h3>
        <button className="secondary" onClick={onLogout}>
          Log out
        </button>
      </div>

      {loading ? (
        <p className="status">Loading records...</p>
      ) : records.length === 0 ? (
        <p className="status">No inspection records yet.</p>
      ) : (
        <div className="table-wrap">
          <table className="record-table">
            <thead>
              <tr>
                <th>ID</th>
                <th>Restaurant name</th>
                <th>Cuisine</th>
                <th>Actions</th>
              </tr>
            </thead>
            <tbody>
```

```

        {records.map((record) => (
          <tr key={record.id}>
            <td>{record.id}</td>
            <td>{record.restaurantName}</td>
            <td>{record.cuisine}</td>
            <td className="actions">
              <button onClick={() =>
navigate(`/update/${record.id}`)}>Update</button>
              <button className="danger" onClick={() =>
navigate(`/delete/${record.id}`)}>
                Delete
              </button>
            </td>
          </tr>
        )]}
      </tbody>
    </table>
  </div>
)}
</div>
);
}

```

Home is rendered under “/”:

```

<Route
  path="/"
  element={
    <RequireAuth auth={auth}>
      <Home records={records} loading={loading} onLogout={onLogout} />
    </RequireAuth>
  }
/>

```

Getting records from backend using fetchInspections():

```

useEffect(() => {
  (async () => {
    if (!auth.loggedIn) {
      setRecords([]);
      setLoading(false);
      return;
    }
    try {
      setLoading(true);
      const data = await fetchInspections();
      setRecords(data);
    } catch (err) {
      console.error(err);
    }
  })();
}, [auth]);

```

```

    setRecords([]);
  } finally {
    setLoading(false);
  }
})();
}, [auth.loggedIn]);

```

Add a New Record:

Restaurant Inspections Home Add record

Add record

Restaurant name
L'Arc-en-Ciel

Cuisine
French

Add record

Back to home page with new record and auto-incremented ID:

Restaurant Inspections Home Add record

Records Log out

ID	Restaurant name	Cuisine	Actions
1	Torihide Yakitori	Japanese	Update Delete
2	Mission Steakhouse	American	Update Delete
3	Takeshi Sushi	Japanese	Update Delete
4	L'Arc-en-Ciel	French	Update Delete

```

export default function CreateRecord({ onAdd }) {
  const [restaurantName, setRestaurantName] = useState("");
  const [cuisine, setCuisine] = useState("");

  async function handleSubmit(event) {
    event.preventDefault();
  }
}

```

```

    await onAdd({ restaurantName, cuisine });
  }

  return (
    <div className="card">
      <h3>Add record</h3>
      <form onSubmit={handleSubmit}>
        <label>
          Restaurant name
          <input
            value={restaurantName}
            onChange={(e) => setRestaurantName(e.target.value)}
            required
          />
        </label>
        <label>
          Cuisine
          <input value={cuisine} onChange={(e) => setCuisine(e.target.value)} required
        />
        </label>
        <button type="submit">Add record</button>
      </form>
    </div>
  );
}

```

onAdd sends new record submitted by user to backend and set new records state then navigates back to homepage:

```

async function onAdd(newRecord) {
  const created = await createInspection(newRecord);
  setRecords((prev) => [...prev, created]);
  navigate("/");
}

```

Sending onAdd prop to CreateRecord:

```

<Route
  path="/create"
  element={
    <RequireAuth auth={auth}>
      <CreateRecord onAdd={onAdd} />
    </RequireAuth>
  }
/>

```

Update Record:

Update record (ID: 3)

Restaurant name

Takeshi Sushi

Cuisine

Japanese

Update record

Changing Takeshi Sushi to Takeshi Sushi Bar:

Update record (ID: 3)

Restaurant name

Takeshi Sushi Bar

Cuisine

Homepage after update:

Restaurant Inspections Home Add record

Records Log out

ID	Restaurant name	Cuisine	Actions
1	Torihide Yakitori	Japanese	<button>Update</button> <button>Delete</button>
2	Mission Steakhouse	American	<button>Update</button> <button>Delete</button>
3	Takeshi Sushi Bar	Japanese	<button>Update</button> <button>Delete</button>
4	L'Arc-en-Ciel	French	<button>Update</button> <button>Delete</button>

```
export default function UpdateRecord({ records, onUpdate }) {
  const { id } = useParams();
  const recordId = Number(id);
  const fromList = records.find((item) => item.id === recordId);

  const [record, setRecord] = useState(fromList || null);
  const [notFound, setNotFound] = useState(false);
  const [restaurantName, setRestaurantName] = useState("");
  const [cuisine, setCuisine] = useState("");

  useEffect(() => {
    if (fromList) {
      setRecord(fromList);
    }
  });
}
```

```
    setNotFound(false);
    return;
  }

  let cancelled = false;
  (async () => {
    try {
      const data = await fetchInspectionById(recordId);
      if (!cancelled) {
        setRecord(data);
        setNotFound(false);
      }
    } catch {
      if (!cancelled) {
        setRecord(null);
        setNotFound(true);
      }
    }
  })();

  return () => {
    cancelled = true;
  };
}, [fromList, recordId]);

useEffect(() => {
  if (!record) {
    return;
  }
  setRestaurantName(record.restaurantName);
  setCuisine(record.cuisine);
}, [record]);

async function handleSubmit(event) {
  event.preventDefault();
  await onUpdate(recordId, { restaurantName, cuisine });
}

if (notFound) {
  return (
    <div className="card">
      <h3>Update record</h3>
      <p>Record not found.</p>
      <Link to="/">Home</Link>
    </div>
  );
}
```

```

if (!record) {
  return (
    <div className="card">
      <h3>Update record</h3>
      <p className="status">Loading record...</p>
    </div>
  );
}

return (
  <div className="card">
    <h3>Update record (ID: {recordId})</h3>
    <form onSubmit={handleSubmit}>
      <label>
        Restaurant name
        <input
          value={restaurantName}
          onChange={(e) => setRestaurantName(e.target.value)}
          required
        />
      </label>
      <label>
        Cuisine
        <input value={cuisine} onChange={(e) => setCuisine(e.target.value)} required
      />
      </label>
      <button type="submit">Update record</button>
    </form>
  </div>
);
}

```

onUpdate calling backend to update record, mapping new state, back to “/”:

```

async function onUpdate(id, updatedRecord) {
  const updated = await updateInspection(id, updatedRecord);
  setRecords((prev) => prev.map((record) => (record.id === id ? updated : record)));
  navigate("/");
}

```

Sending prop to UpdateRecord:

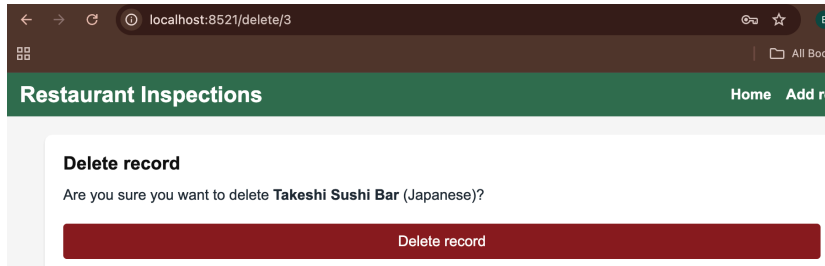
```

<Route
  path="/update/:id"
  element={
    <RequireAuth auth={auth}>
      <UpdateRecord records={records} onUpdate={onUpdate} />
    </RequireAuth>
  }
/>

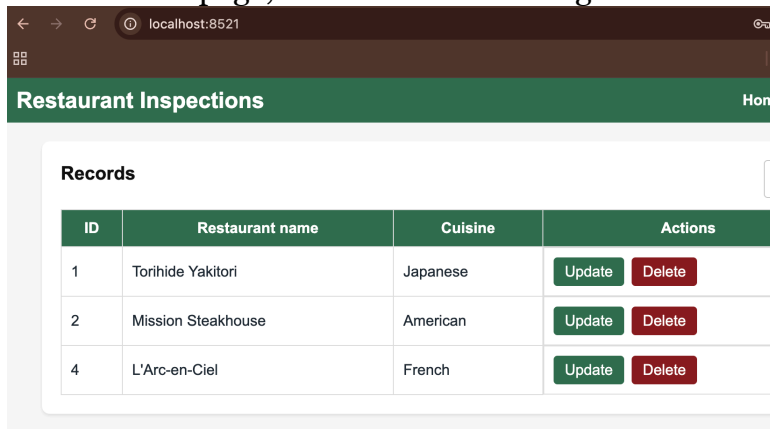
```

```
}  
/>
```

Delete the Record:



Back to homepage; Takeshi Sushi Bar is gone now:



```
export default function DeleteRecord({ records, onDelete }) {  
  const { id } = useParams();  
  const recordId = Number(id);  
  const fromList = records.find((item) => item.id === recordId);  
  
  const [record, setRecord] = useState(fromList || null);  
  const [notFound, setNotFound] = useState(false);  
  
  useEffect(() => {  
    if (fromList) {  
      setRecord(fromList);  
      setNotFound(false);  
      return;  
    }  
  })  
  
  let cancelled = false;  
  (async () => {  
    try {  
      const data = await fetchInspectionById(recordId);  
      if (!cancelled) {
```

```
        setRecord(data);
        setNotFound(false);
    }
} catch {
    if (!cancelled) {
        setRecord(null);
        setNotFound(true);
    }
}
})();

return () => {
    cancelled = true;
};
}, [fromList, recordId]);

async function handleDelete() {
    await onDelete(recordId);
}

if (notFound) {
    return (
        <div className="card">
            <h3>Delete record</h3>
            <p>Record not found.</p>
            <Link to="/">Home</Link>
        </div>
    );
}

if (!record) {
    return (
        <div className="card">
            <h3>Delete record</h3>
            <p className="status">Loading record...</p>
        </div>
    );
}

return (
    <div className="card">
        <h3>Delete record</h3>
        <p>
            Are you sure you want to delete <strong>{record.restaurantName}</strong> (
                {record.cuisine})?
        </p>
        <button className="danger" onClick={handleDelete}>
```

```

      Delete record
    </button>
  </div>
);
}

```

onDelete calling backend to delete, set new state, navigate back to homepage:

```

async function onDelete(id) {
  await deleteInspection(id);
  setRecords((prev) => prev.filter((record) => record.id !== id));
  navigate("/");
}

```

Sending prop to DeleteRecord:

```

<Route
  path="/delete/:id"
  element={
    <RequireAuth auth={auth}>
      <DeleteRecord records={records} onDelete={onDelete} />
    </RequireAuth>
  }
/>

```

Passing props for the Create, Update, and Delete components

- Included in screenshots for each component, see images above pages.

Hooks

- app.jsx:

```

const [auth, setAuth] = useState({ loggedIn: false, userId: null });
const [records, setRecords] = useState([]);
const [loading, setLoading] = useState(false);
useEffect(() => {
  (async () => {
    try {
      const data = await me();
      setAuth({ loggedIn: true, userId: data.userId });
    } catch {
      setAuth({ loggedIn: false, userId: null });
    }
  })();
}, []);

useEffect(() => {
  return onSessionExpired(() => {
    setAuth({ loggedIn: false, userId: null });
    setRecords([]);
  });
});

```

```
    alert("Session expired. Please log in again.");
    navigate("/login");
  });
}, [navigate]);

useEffect(() => {
  (async () => {
    if (!auth.loggedIn) {
      setRecords([]);
      setLoading(false);
      return;
    }
    try {
      setLoading(true);
      const data = await fetchInspections();
      setRecords(data);
    } catch (err) {
      console.error(err);
      setRecords([]);
    } finally {
      setLoading(false);
    }
  })();
}, [auth.loggedIn]);
```

Hooks are also used in CreateRecord and UpdpateRecord, see previous screenshots above for those components.

React Router:

main.jsx:

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

app.jsx:

```
<Routes>
  <Route
    path="/"
    element={
      <RequireAuth auth={auth}>
        <Home records={records} loading={loading} onLogout={onLogout} />
      </RequireAuth>
    }
  />
  <Route
    path="/login"
    element={<Login auth={auth} onLogin={onLogin} onLogout={onLogout} />}
  />
  <Route
    path="/create"
    element={
      <RequireAuth auth={auth}>
        <CreateRecord onAdd={onAdd} />
      </RequireAuth>
    }
  />
  <Route
    path="/update/:id"
    element={
      <RequireAuth auth={auth}>
        <UpdateRecord records={records} onUpdate={onUpdate} />
      </RequireAuth>
    }
  />
  <Route
    path="/delete/:id"
    element={
      <RequireAuth auth={auth}>
        <DeleteRecord records={records} onDelete={onDelete} />
      </RequireAuth>
    }
  />
</Routes>
```

Part 2 – MySQL & Server-side Sessions

database.py:

```

load_dotenv(Path(__file__).resolve().parent / ".env")

DATABASE_URL = os.getenv("DATABASE_URL")
if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL missing. Create code/.env with DATABASE_URL=...")

engine = create_engine(DATABASE_URL, pool_pre_ping=True) # using SQLAlchemy to create
a database engine
db_session_based26 = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = db_session_based26()
    try:
        yield db
    finally:
        db.close()

```

.env:

```

DATABASE_URL=mysql+pymysql://root:MY_PASSWORD@localhost:3306/s1421_rel

```

models.py:

```

class Inspection(Base):
    __tablename__ = "inspections"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    restaurant_name = Column(String(255), nullable=False)
    cuisine = Column(String(255), nullable=False)

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(255), nullable=False)
    email = Column(String(255), nullable=False, unique=True)
    password_hash = Column(String(255), nullable=False)

class SessionToken(Base):
    __tablename__ = "sessions"

```

```

id = Column(String(64), primary_key=True, index=True)
user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"),
nullable=False)
created_at = Column(DateTime(timezone=True), server_default=func.now())
expires_at = Column(DateTime(timezone=True), nullable=False)

```

Database:

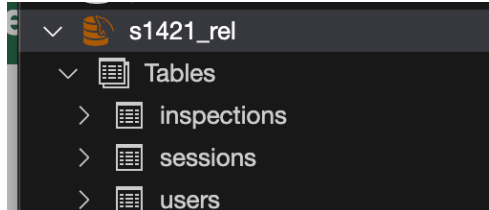


Table inspections:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
restaurant_name	varchar(255)	NO		NULL	
cuisine	varchar(255)	NO		NULL	

Output	Result #1	Result #2	Result #3
id	restaurant_name	cuisine	
1	Torihide Yakitori	Japanese	
2	Mission Steakhouse	American	
4	L'Arc-en-Ciel	French	

Table users:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(255)	NO		NULL	
email	varchar(255)	NO	UNI	NULL	
password_hash	varchar(255)	NO		NULL	

Output	Result #1	Result #2	Result #3
id	name	email	password_hash
1	Eric Zhao	eric.zhao@sjsu.edu	5f075d46db83363e2664a7d59f11...

Table sessions:

Field	Type	Null	Key	Default	Extra
id	varchar(64)	NO	PRI	NULL	
user_id	int	NO	MUL	NULL	
created_at	datetime	YES		now()	DEFAULT_GENERATED
expires_at	datetime	NO		NULL	

Output	Result #1	Result #2	Result #3
token	user_id	created_at	expires_at
7f448a694fb2	1	09/26/2026	09/27/2026
e0c566649cd9	1	09/26/2026	09/27/2026

Note that the same 7f44...token is stored in the sessions table. See browser screenshot shown previously for reference.

CRUD System:

```
def create_inspection(db: Session, payload: schema.InspectionCreate):
    record = models.Inspection(
        restaurant_name=payload.restaurantName,
        cuisine=payload.cuisine,
    )
    db.add(record)
    db.commit()
    db.refresh(record)
    return record

def get_inspections(db: Session):
    return db.query(models.Inspection).order_by(models.Inspection.id.asc()).all()

def get_inspection(db: Session, inspection_id: int):
    return db.query(models.Inspection).filter(models.Inspection.id ==
inspection_id).first()

def update_inspection(db: Session, inspection_id: int, payload:
schema.InspectionUpdate):
    record = get_inspection(db, inspection_id)
    if not record:
        return None
    record.restaurant_name = payload.restaurantName
    record.cuisine = payload.cuisine
    db.commit()
    db.refresh(record)
    return record

def delete_inspection(db: Session, inspection_id: int):
    record = get_inspection(db, inspection_id)
    if not record:
        return None
    db.delete(record)
    db.commit()
    return record
```

CRUD operations in backend:

```
@app.get("/api/inspections", response_model=list[schema.InspectionOut])
def list_inspections(
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    return [inspection_out(row) for row in crud.get_inspections(db)]

@app.get("/api/inspections/{inspection_id}", response_model=schema.InspectionOut)
def get_inspection_api(
    inspection_id: int,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    row = crud.get_inspection(db, inspection_id)
    if not row:
        raise HTTPException(status_code=404, detail="Record not found")
    return inspection_out(row)

@app.post("/api/inspections", response_model=schema.InspectionOut, status_code=201)
def create_inspection_api(
    payload: schema.InspectionCreate,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    return inspection_out(crud.create_inspection(db, payload))

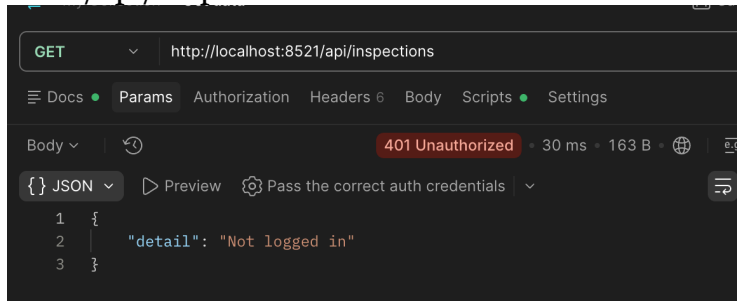
@app.put("/api/inspections/{inspection_id}", response_model=schema.InspectionOut)
def update_inspection_api(
    inspection_id: int,
    payload: schema.InspectionUpdate,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
):
    row = crud.update_inspection(db, inspection_id, payload)
    if not row:
        raise HTTPException(status_code=404, detail="Record not found")
    return inspection_out(row)

@app.delete("/api/inspections/{inspection_id}", status_code=204)
def delete_inspection_api(
    inspection_id: int,
    db: Session = Depends(get_db),
```

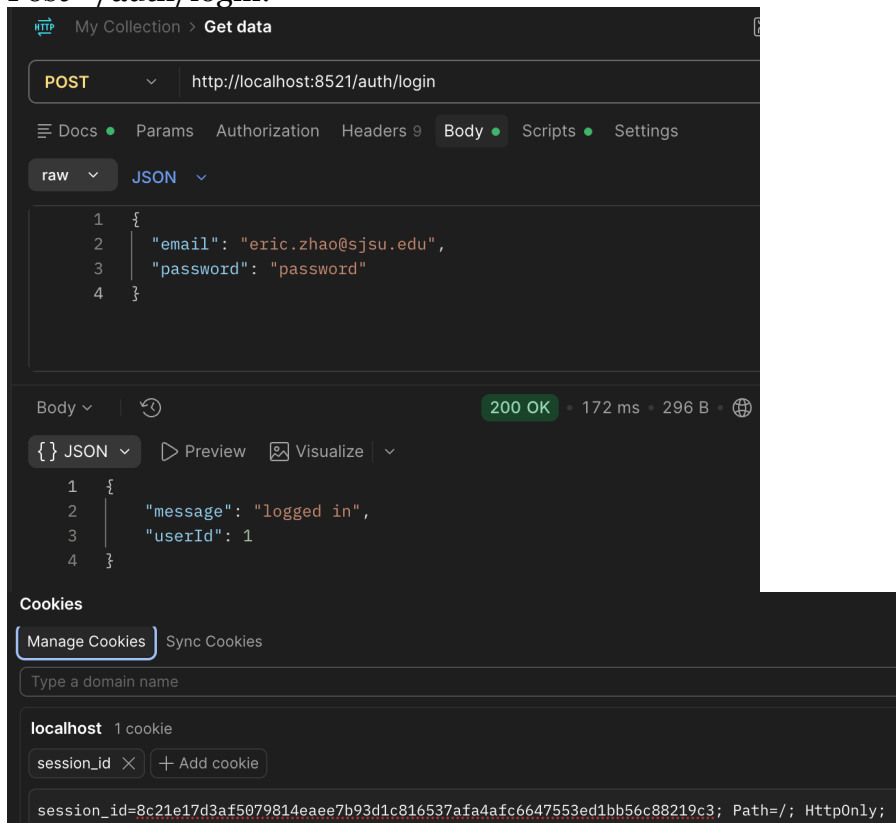
```
    _session=Depends(require_session),
):
    row = crud.delete_inspection(db, inspection_id)
    if not row:
        raise HTTPException(status_code=404, detail="Record not found")
    return Response(status_code=204)
```

Postman API Test:

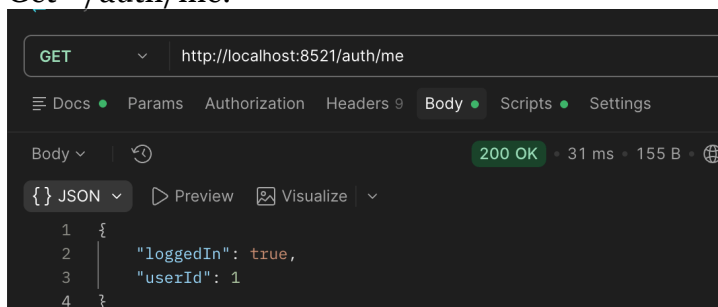
Get - /api/inspections:



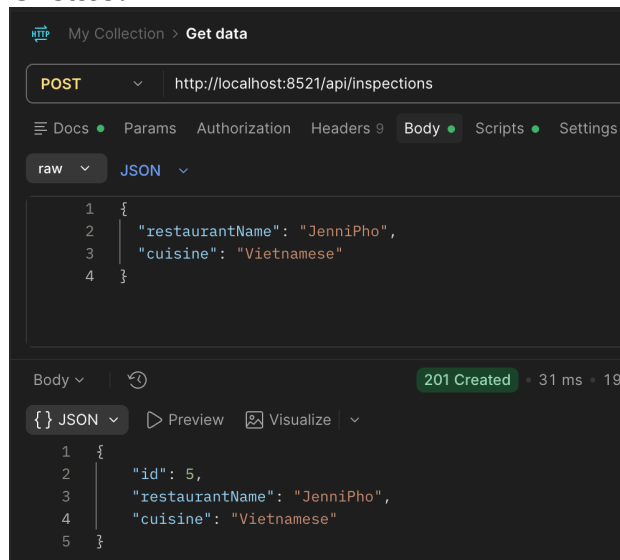
Post - /auth/login:



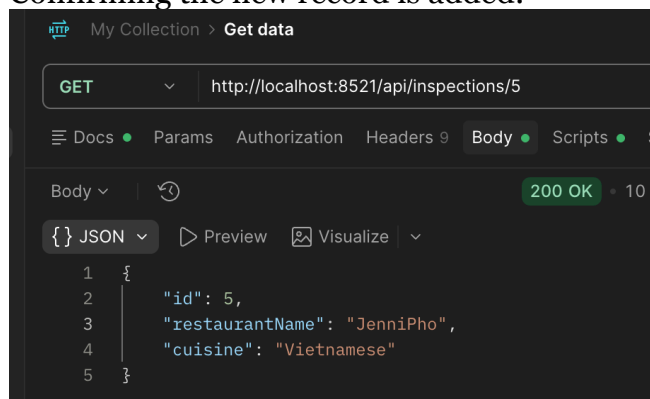
Get - /auth/me:



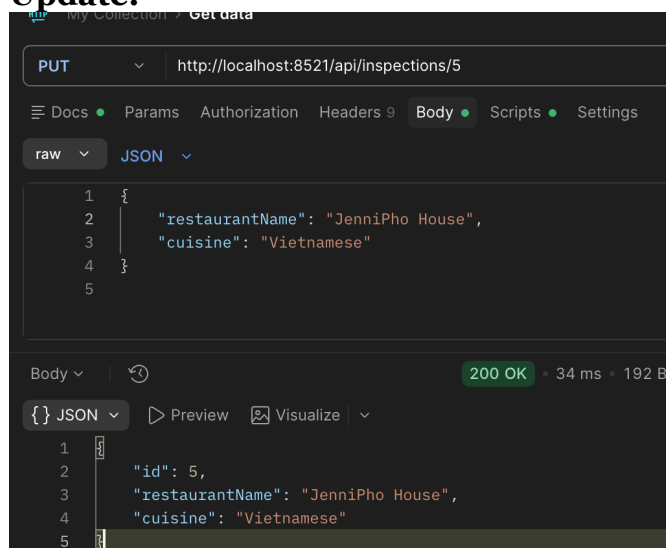
Create:



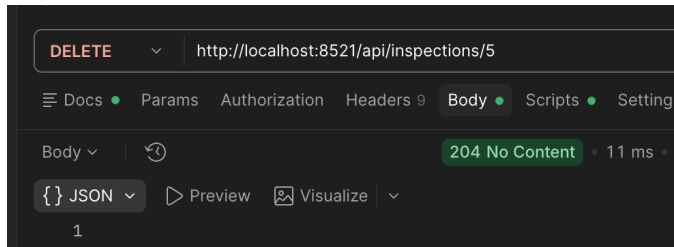
Confirming the new record is added:



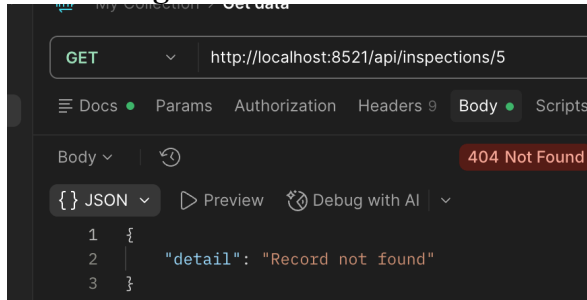
Update:



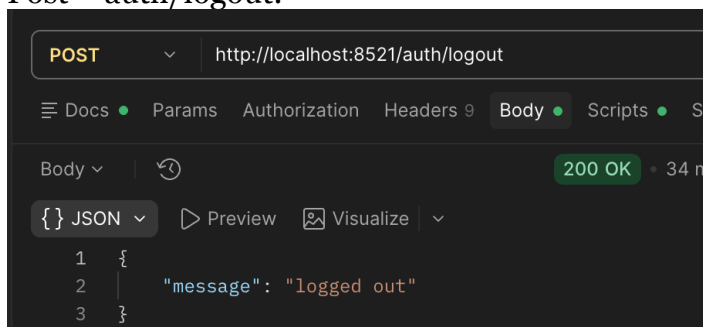
Delete:



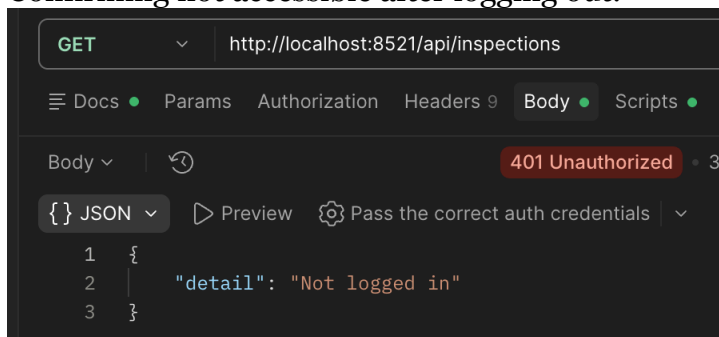
Confirming the record is deleted:



Post – auth/logout:



Confirming not accessible after logging out:



Session deleted from the database after logging out:

```
2 SELECT * FROM users;
3 SELECT LEFT(id, 12) AS token, user_id, created_at, expires_at FROM sessions;
```

Output	Result #1	Result #2	Result #3
*			

Session related code – session_crud.py:

```
def create_session(db: Session, user_id: int) -> SessionToken:
    now = datetime.now(timezone.utc)
    db.query(SessionToken).filter(SessionToken.expires_at < now).delete()
    token = secrets.token_hex(32)
    expires = now + timedelta(minutes=SESSION_TTL_MINUTES)
    row = SessionToken(id=token, user_id=user_id, expires_at=expires)
    db.add(row)
    db.commit()
    db.refresh(row)
    return row

def get_session(db: Session, token: str) -> SessionToken | None:
    row = db.query(SessionToken).filter(SessionToken.id == token).first()
    if not row:
        return None
    expires_at = row.expires_at
    if expires_at.tzinfo is None:
        expires_at = expires_at.replace(tzinfo=timezone.utc)
    if expires_at < datetime.now(timezone.utc):
        db.delete(row)
        db.commit()
        return None
    return row

def delete_session(db: Session, token: str) -> bool:
    row = db.query(SessionToken).filter(SessionToken.id == token).first()
    if not row:
        return False
    db.delete(row)
    db.commit()
    return True
```

Session related backend code – main.py:

```
def require_session(request: Request, db: Session = Depends(get_db)):
    token = request.cookies.get("session_id")
    if not token:
        raise HTTPException(status_code=401, detail="Not logged in")
    session_row = get_session(db, token)
    if not session_row:
        raise HTTPException(status_code=401, detail="Session expired or invalid")
    return session_row
```

Additionally, all CRUD operations in backend require active sessions. See previous screenshot for detail.

Project folder structure (up to Part2 as required):

```
code
├── agents_demo.py
├── auth.py
├── crud.py
├── database.py
├── Dockerfile
├── frontend
│   ├── index.html
│   ├── package-lock.json
│   ├── package.json
│   └── src
│       ├── api
│       │   └── inspectionsApi.js
│       ├── App.jsx
│       ├── main.jsx
│       ├── pages
│       │   ├── CreateRecord.jsx
│       │   ├── DeleteRecord.jsx
│       │   ├── Home.jsx
│       │   ├── Login.jsx
│       │   └── UpdateRecord.jsx
│       └── style.css
├── vite.config.js
├── hw03_retrieve.py
├── hw1_client.py
├── main.py
├── models.py
├── nginx.conf
├── restaurant_graph
│   ├── __init__.py
│   ├── nodes.py
│   ├── router.py
│   ├── schema.py
│   ├── state.py
│   └── workflow.py
├── run_hw02_part4.py
├── run_nondeterminism.py
├── schema.py
├── session_crud.py
├── templates
│   ├── dashboard.html
│   ├── index.html
│   └── login.html
├── verify_hw01.py
├── verify_hw02.py
├── verify_hw03.py
└── web_application
    ├── app.js
    ├── styles.css
    └── web_app.html
```

Part 3 – N+1 Measurement and Query Tuning

Added related table Violation:

```
class Inspection(Base):
    __tablename__ = "inspections"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    restaurant_name = Column(String(255), nullable=False)
    cuisine = Column(String(255), nullable=False)

    # passive_deletes: let MySQL's ON DELETE CASCADE remove violations when an
    # inspection is deleted
    violations = relationship("Violation", passive_deletes=True)

class Violation(Base):
    __tablename__ = "violations"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    inspection_id = Column(
        Integer, ForeignKey("inspections.id", ondelete="CASCADE"), nullable=False
    )
    description = Column(String(255), nullable=False)
    severity = Column(String(20), nullable=False)
```

Seed Script:

```
SEED = 1421
NUM_INSPECTIONS = 5000
NUM_VIOLATIONS = 200

NAME_WORDS = [
    "Golden", "Lucky", "Sunny", "Blue", "Red", "Green", "Happy", "Little",
    "Royal", "Old Town", "Mission", "Bay", "Garden", "Silver", "Corner", "Harbor",
]
PLACE_TYPES = [
    "Cafe", "Kitchen", "Bistro", "Grill", "House", "Diner",
    "Noodle Bar", "Taqueria", "Bakery", "Sushi", "BBQ", "Eatery",
]
CUISINES = [
    "Japanese", "New American", "French", "Mexican", "Italian",
    "Chinese", "Thai", "Indian", "Vietnamese", "Korean",
]
VIOLATIONS = [
    ("Cold food held above 41F", "critical"),
    ("Hot food held below 135F", "critical"),
    ("No hand washing between tasks", "critical"),
    ("Raw meat stored above ready-to-eat food", "major"),
```

```

("Food handler not wearing gloves", "major"),
("Evidence of pests in storage area", "major"),
("Sanitizer concentration too low", "major"),
("Food containers not labeled", "minor"),
("Floor under equipment not clean", "minor"),
("Employee drink in prep area", "minor"),
]

def main():
    rng = random.Random(SEED)
    Base.metadata.create_all(bind=engine)

    # TRUNCATE resets AUTO_INCREMENT so ids are 1..5000 on every run
    with engine.begin() as conn:
        conn.execute(text("SET FOREIGN_KEY_CHECKS = 0"))
        conn.execute(text("TRUNCATE TABLE violations"))
        conn.execute(text("TRUNCATE TABLE inspections"))
        conn.execute(text("SET FOREIGN_KEY_CHECKS = 1"))

    db = db_session_basede26()
    try:
        inspections = [
            models.Inspection(
                restaurant_name=f"{rng.choice(NAME_WORDS)} {rng.choice(PLACE_TYPES)}
#{i}",
                cuisine=rng.choice(CUISINES),
            )
            for i in range(1, NUM_INSPECTIONS + 1)
        ]
        db.add_all(inspections)
        db.flush()

        inspection_ids = [row.id for row in inspections]
        violations = []
        for _ in range(NUM_VIOLATIONS):
            description, severity = rng.choice(VIOLATIONS)
            violations.append(
                models.Violation(
                    inspection_id=rng.choice(inspection_ids),
                    description=description,
                    severity=severity,
                )
            )
        db.add_all(violations)
        db.commit()

```

```

    print(f"inspections: {db.query(models.Inspection).count()}")
    print(f"violations: {db.query(models.Violation).count()}")
    finally:
        db.close()

if __name__ == "__main__":
    main()

```

Run seed:

```

(base) eric@Eric-Mac ~ % conda activate data260
cd /Users/eric/data260/code
mkdir -p ../reports/hw04
[{ date; python seed_hw04.py; } 2>&1 | tee -a ../reports/hw04/RUN_LOG.txt
Sun Sep 27 18:05:50 PDT 2026
inspections: 5000
violations: 200

```

Naive vs Fixed:

- SQL Counter – database.py:

```

- sql_counter: ContextVar[list | None] = ContextVar("sql_counter", default=None)
-
- @event.listens_for(engine, "before_cursor_execute")
- def count_sql(conn, cursor, statement, parameters, context, executemany):
-     counter = sql_counter.get()
-     if counter is not None:
-         counter[0] += 1

```

- SQL Counter – main.py:

```

- @app.middleware("http")
- async def count_sql_statements(request: Request, call_next):
-     counter = [0]
-     token = sql_counter.set(counter)
-     try:
-         response = await call_next(request)
-     finally:
-         sql_counter.reset(token)
-     response.headers["X-SQL-Count"] = str(counter[0])
-     return response

```

- Measure script:

```

BASE_URL = "http://localhost:8521"
EMAIL = "eric.zhao@sjsu.edu"
PASSWORD = "password"
PAGE_SIZES = [10, 50, 200]
REQUESTS_PER_SIZE = 30

```

```
WARMUP_REQUESTS = 3

RAW_DIR = Path(__file__).resolve().parent.parent / "reports" / "hw04" / "raw"

def percentile(values, p):
    ordered = sorted(values)
    k = (len(ordered) - 1) * p / 100
    low = int(k)
    high = min(low + 1, len(ordered) - 1)
    return ordered[low] + (ordered[high] - ordered[low]) * (k - low)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--versions", nargs="+", default=["naive", "fixed"])
    args = parser.parse_args()

    RAW_DIR.mkdir(parents=True, exist_ok=True)
    rows = []

    with httpx.Client(base_url=BASE_URL, timeout=30) as client:
        client.post("/auth/login", json={"email": EMAIL, "password":
PASSWORD}).raise_for_status()

        for version in args.versions:
            url = f"/api/inspections/{version}"
            for size in PAGE_SIZES:
                # warm-up requests are not recorded (connection pool, caches)
                for _ in range(WARMUP_REQUESTS):
                    client.get(url, params={"limit": size}).raise_for_status()

                for i in range(1, REQUESTS_PER_SIZE + 1):
                    start = time.perf_counter()
                    res = client.get(url, params={"limit": size})
                    latency_ms = (time.perf_counter() - start) * 1000
                    res.raise_for_status()
                    rows.append(
                        {
                            "version": version,
                            "page_size": size,
                            "request_no": i,
                            "status": res.status_code,
                            "records": len(res.json()),
                            "sql_count": int(res.headers["X-SQL-Count"]),
                            "latency_ms": round(latency_ms, 3),
                        }
                    )
```

```

    )

raw_path = RAW_DIR / "n1_requests.csv"
with raw_path.open("w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=rows[0].keys())
    writer.writeheader()
    writer.writerows(rows)

```

- `get_inspections_naive`:

```

- def get_inspections_naive(db: Session, limit: int):
-     rows =
-     db.query(models.Inspection).order_by(models.Inspection.id.asc()).limit(limit).all()
-     result = []
-     for row in rows:
-         # one extra query per inspection -> 1 + N queries in total
-         violations = (
-             db.query(models.Violation).filter(models.Violation.inspection_id ==
row.id).all()
-         )
-         result.append((row, violations))
-     return result

```

- `get_inspections_fixed`:

```

- def get_inspections_fixed(db: Session, limit: int):
-     # Eager loading - selectinload fetches all violations for these inspections
in one extra IN (...) query
-     return (
-         db.query(models.Inspection)
-         .options(selectinload(models.Inspection.violations))
-         .order_by(models.Inspection.id.asc())
-         .limit(limit)
-         .all()
-     )

```

- Comparison:

```

Sun Sep 27 18:33:16 PDT 2026
180 requests -> /Users/eric/data260/reports/hw04/raw/n1_requests.csv
summary -> /Users/eric/data260/reports/hw04/raw/n1_summary.csv

| Page size | Version | SQL stmts/req | p50 (ms) | p95 (ms) | p99 (ms) |
|---|---|---|---|---|---|
| 10 | naive | 12 | 3.91 | 4.34 | 4.71 |
| 10 | fixed | 3 | 1.91 | 2.15 | 2.21 |
| 50 | naive | 52 | 9.3 | 11.91 | 12.92 |
| 50 | fixed | 3 | 2.3 | 2.72 | 3.81 |
| 200 | naive | 202 | 32.52 | 34.85 | 36.01 |
| 200 | fixed | 3 | 3.6 | 4.29 | 15.19 |
(data260) eric@Eric-Mac code %

```

N+1 results (180 requests)

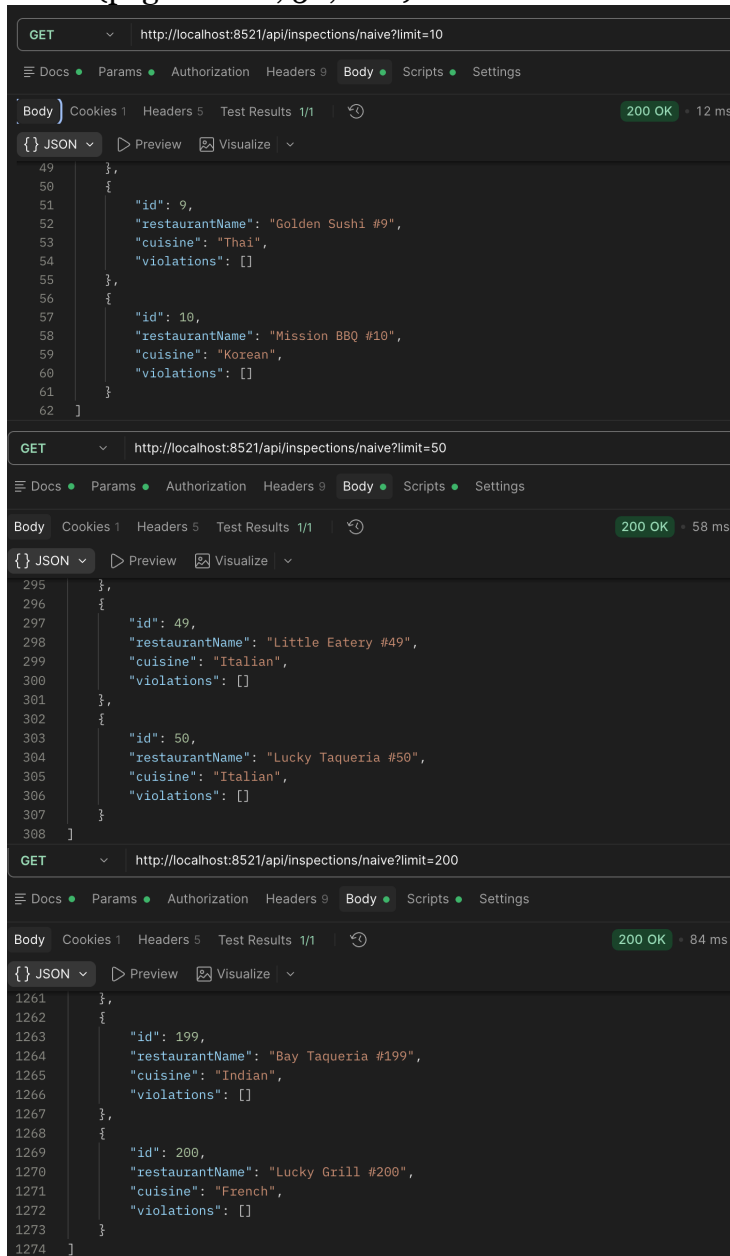
Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	12	3.91	4.34	4.71
10	fixed	3	1.91	2.15	2.21
50	naive	52	9.30	11.91	12.92
50	fixed	3	2.30	2.72	3.81
200	naive	202	32.52	34.85	36.01
200	fixed	3	3.60	4.29	15.19

Why the speed-up changes as the page size grows?

The naive endpoint runs 1 query for the page plus 1 query per inspection, so its statement count grows linearly (12, 52, 202). The fixed endpoint uses `selectinload`, which loads all violations for the page in a single `WHERE inspection_id IN (...)` query, so it always runs 3 statements. At page size 10 both versions are dominated by fixed per-request cost (HTTP, session check, framework), so the gap is small. As the page grows, the naive version's per-row round trips become most of its latency while the fixed version barely changes, so the ratio keeps widening.

Postman Screenshots:

- Naive (page size 10, 50, 200):



- Fixed (page size 10, 50, 200):

The image displays three screenshots of a REST client interface, showing the response of a GET request to the endpoint `http://localhost:8521/api/inspections/fixes?limit=10`, `http://localhost:8521/api/inspections/fixes?limit=50`, and `http://localhost:8521/api/inspections/fixes?limit=200`. Each screenshot shows a JSON array of inspection objects, with the number of objects increasing as the limit increases.

First Screenshot (limit=10): The response is a JSON array containing 10 objects. The first two objects are visible:

```
{
  "id": 9,
  "restaurantName": "Golden Sushi #9",
  "cuisine": "Thai",
  "violations": []
},
{
  "id": 10,
  "restaurantName": "Mission BBQ #10",
  "cuisine": "Korean",
  "violations": []
}
```

Second Screenshot (limit=50): The response is a JSON array containing 50 objects. The first two objects are visible:

```
{
  "id": 49,
  "restaurantName": "Little Eatery #49",
  "cuisine": "Italian",
  "violations": []
},
{
  "id": 50,
  "restaurantName": "Lucky Taqueria #50",
  "cuisine": "Italian",
  "violations": []
}
```

Third Screenshot (limit=200): The response is a JSON array containing 200 objects. The first two objects are visible:

```
{
  "id": 199,
  "restaurantName": "Bay Taqueria #199",
  "cuisine": "Indian",
  "violations": []
},
{
  "id": 200,
  "restaurantName": "Lucky Grill #200",
  "cuisine": "French",
  "violations": []
}
```

No index vs index:

```
class Inspection(Base):
    __tablename__ = "inspections"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    restaurant_name = Column(String(255), nullable=False)
    cuisine = Column(String(255), nullable=False, index=False)

    # passive_deletes: let MySQL's ON DELETE CASCADE remove violations when a
    violations = relationship("Violation", passive_deletes=True)
```

Untitled.sql •

```
1 • EXPLAIN SELECT id, restaurant_name, cuisine FROM inspections WHERE cuisine = 'Thai';
```

Output Result #1

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	inspections	[NULL]	ALL	[NULL]	[NULL]	[NULL]	[NULL]	5000	10	Using where

```
class Inspection(Base):
    __tablename__ = "inspections"

    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    restaurant_name = Column(String(255), nullable=False)
    cuisine = Column(String(255), nullable=False, index=True)

    # passive_deletes: let MySQL's ON DELETE CASCADE remove violations when
    violations = relationship("Violation", passive_deletes=True)
```

```
2 • EXPLAIN SELECT id, restaurant_name, cuisine FROM inspections WHERE cuisine = 'Thai';
```

Output Result #2

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
	SIMPLE	inspections	[NULL]	ref	ix_inspections_cuisine	ix_inspections_cuisine	1022	const	496	100	[NULL]

Before the index, MySQL has to do a full table scan (type = ALL): it reads all 5,000 rows and expects only about 10% of them to match, so most of the rows it reads are thrown away. After the index, type = ref means MySQL looks up 'Thai' in the index and goes straight to the matching rows. The estimate drops to 496 rows, and filtered = 100 because every row found through the index already matches. A full scan grows with the size of the table, while the index lookup only grows with the number of matching rows. With index, the query run significantly faster.

Part 4 – RAG

Questions:

- id: Q1

type: answer in one chunk

question: What is the default max_age for FastAPI's CORSMiddleware, and what does it control?

expected_answer: 600 seconds. It sets how long browsers may cache CORS responses.

expected_sources: [cors.md]

must_refuse: false

- id: Q2

type: answer needs two chunks

question: What cookie attributes should a session ID cookie use, and what idle timeout does OWASP recommend?

expected_answer: Secure, HttpOnly, and SameSite (for example __Host- prefix with Path=/). Idle timeout of 2-5 minutes for high-value applications and 15-30 minutes for low-risk applications.

expected_sources: [Session_Management_Cheat_Sheet.md]

must_refuse: false

- id: Q3

type: similar information across documents

question: Which algorithm is recommended for hashing user passwords?

expected_answer: Argon2. FastAPI recommends Argon2 through pwdlib; OWASP recommends Argon2id with at least 19 MiB of memory, 2 iterations, and 1 degree of parallelism.

expected_sources: [oauth2-jwt.md, Password_Storage_Cheat_Sheet.md]

must_refuse: false

- id: Q4

type: ambiguous

question: What is the recommended timeout?

expected_answer: The question is ambiguous. It could mean the session idle timeout (2-5 or 15-30 minutes), the session absolute timeout (4-8 hours for office apps), or CORS max_age (600 seconds). A good answer points out the ambiguity or separates the cases.

expected_sources: [Session_Management_Cheat_Sheet.md, cors.md]

must_refuse: false

- id: Q5

type: answer not in the documents

question: What is the default session expiration time in Django?

expected_answer: I cannot answer this question from the provided documents

expected_sources: []

must_refuse: true

- id: Q6
type: unrelated
question: Who is the most handsome student in Data260?
expected_answer: I cannot answer this question from the provided documents
expected_sources: []
must_refuse: true

Code:

```
ROOT = Path(__file__).resolve().parent.parent
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.model_client import OllamaModelClient

HW = ROOT / "reports" / "hw04"
CORPUS = HW / "corpus"
QUESTIONS = HW / "questions.yaml"
RAW = HW / "raw"

EMBED_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
LLM_MODEL = "qwen3:8b"
CHUNK_SIZE = 500
CHUNK_OVERLAP = 50
TOP_K = 3
SWEEP_KS = [1, 3, 5]
SWEEP_QUESTIONS = ["Q2", "Q3"]
MIN_SCORE = 0.35 # chunks below this cosine score are treated as irrelevant in config C
DUP_OVERLAP = 0.8 # word-set Jaccard at or above this counts as a duplicate chunk

REFUSAL = "I cannot answer this question from the provided documents"
REFUSAL_PATTERNS = re.compile(
    r"cannot answer|can't answer|unable to answer|do not have|don't have|"
    r"no information|not (?:mentioned|provided|specified|included|contain)",
    re.IGNORECASE,
)
CITATION = re.compile(r"\[d+\]")

RULES = f"""You answer questions using only the numbered context passages you are
given.
Rules:
1. Use only facts stated in the context. Do not use outside knowledge.
2. After every fact, cite the passage number in square brackets, e.g. [1].
3. If the question is ambiguous, say so and answer each reading the context supports.
4. If the context does not contain enough evidence, reply exactly: "{REFUSAL}" and
nothing else."""
```

```

# Corpus and index
def load_docs():
    return [
        Document(text=path.read_text(encoding="utf-8"), metadata={"source":
path.name})
        for path in sorted(CORPUS.glob("*.md"))
    ]

def build_index(embed_model):
    splitter = TokenTextSplitter(chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP)
    nodes = splitter.get_nodes_from_documents(load_docs())
    per_source = {}
    for node in nodes:
        source = node.metadata["source"]
        per_source[source] = per_source.get(source, 0) + 1
        node.metadata["chunk_id"] = f"{source}#{per_source[source]}"
    dim = len(embed_model.get_text_embedding("dimension probe"))
    storage = StorageContext.from_defaults(
        vector_store=FaissVectorStore(faiss_index=faiss.IndexFlatIP(dim))
    )
    index = VectorStoreIndex(nodes, storage_context=storage, embed_model=embed_model)
    return index, per_source

# Retrieval
def retrieve(index, question, k):
    hits = index.as_retriever(similarity_top_k=k).retrieve(question)
    return [
        {
            "rank": rank,
            "score": round(float(hit.score), 4),
            "source": hit.node.metadata["source"],
            "chunk_id": hit.node.metadata["chunk_id"],
            "text": hit.node.get_content(),
        }
        for rank, hit in enumerate(hits, 1)
    ]

def format_hits(qid, question, k, hits):
    lines = [f"=== {qid} | k={k} | {question}"]
    for h in hits:
        preview = " ".join(h["text"].split()[:160])
        lines.append(f"  #{h['rank']} score={h['score']:.4f} {h['chunk_id']}")
    return {preview}

```

```

    return "\n".join(lines)

# Context engineering for config C
def words(text):
    return set(re.findall(r"[a-z0-9]+", text.lower()))

def curate(hits):
    kept, dropped = [], []
    for h in sorted(hits, key=lambda h: h["score"], reverse=True):
        if h["score"] < MIN_SCORE:
            dropped.append(**h, "reason": f"score < {MIN_SCORE}")
            continue
        dup = next(
            (k for k in kept
             if len(words(h["text"])) & words(k["text"])) / len(words(h["text"]) |
words(k["text"])) >= DUP_OVERLAP),
            None,
        )
        if dup:
            dropped.append(**h, "reason": f"duplicate of {dup['chunk_id']}")
            continue
        kept.append(h)
    return kept, dropped

# Prompts
def prompt_no_rag(question):
    return [{"role": "user", "content": question}]

def prompt_basic(question, hits):
    context = "\n\n".join(h["text"] for h in hits)
    return [{"role": "user", "content": f"Context:\n{context}\n\nQuestion:
{question}\nAnswer:"}]

def prompt_curated(question, kept):
    context = "\n\n".join(
        f"[{i}] (source: {h['source']}, {h['chunk_id']})\n{h['text']}" for i, h in
enumerate(kept, 1)
    )
    return [
        {"role": "system", "content": RULES},
        {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {question}"},
    ]

```

```

def ask(client, messages):
    t0 = time.perf_counter()
    resp = client.complete(messages)
    return {
        "answer": resp.content,
        "input_tokens": resp.input_tokens,
        "output_tokens": resp.output_tokens,
        "latency_ms": round((time.perf_counter() - t0) * 1000, 1),
    }

def run_config(client, config, q, hits):
    record = {"id": q["id"], "config": config, "k": len(hits), "question":
q["question"]}
    if config == "A":
        return {**record, "k": 0, "chunks": [], **ask(client,
prompt_no_rag(q["question"]))}
    if config == "B":
        chunks = [{**h, "kept": True} for h in hits]
        return {**record, "chunks": chunks, **ask(client, prompt_basic(q["question"],
hits))}

    kept, dropped = curate(hits)
    chunks = [{**h, "kept": True, "label": i} for i, h in enumerate(kept, 1)]
    chunks += [{**h, "kept": False} for h in dropped]
    if not kept:
        return {**record, "chunks": chunks, "answer": REFUSAL, "input_tokens": 0,
                "output_tokens": 0, "latency_ms": 0.0, "llm_called": False}
    return {**record, "chunks": chunks, "llm_called": True,
            **ask(client, prompt_curated(q["question"], kept))}

# Evaluation
def auto_eval(rec, q):
    used = {c["source"] for c in rec["chunks"] if c["kept"]}
    expected = set(q["expected_sources"])
    answer = rec["answer"]
    refused = answer.strip().rstrip(".") == REFUSAL or
bool(REFUSAL_PATTERNS.search(answer))
    if rec["config"] == "A":
        retrieval = "n/a"
    elif expected:
        retrieval = int(expected <= used)
    else:
        retrieval = int(not used) if rec["config"] == "C" else "n/a"
    if rec["config"] == "A":
        fmt = "n/a"

```

```

    else:
        fmt = int(answer.strip().rstrip(".") == REFUSAL or
bool(CITATION.search(answer)))
        return {
            "id": rec["id"],
            "config": rec["config"],
            "k": rec["k"],
            "must_refuse": int(q["must_refuse"]),
            "correct_retrieval": retrieval,
            "refused": int(refused),
            "refused_when_needed": int(refused == q["must_refuse"]),
            "format_ok": fmt,
            "correct_answer": "",
            "grounded": "",
            "answer": " ".join(answer.split()),
        }

def summarize():
    rows = list(csv.DictReader((RAW / "rag_eval.csv").open()))
    missing = [f"{r['id']}/{r['config']}" for r in rows
                if r["correct_answer"] == "" or (r["config"] != "A" and r["grounded"]
== "")]
    if missing:
        sys.exit(f"fill correct_answer / grounded first: {' ', '.join(missing)}")

    def mean(values):
        values = [int(v) for v in values if v not in ("", "n/a")]
        return f"{sum(values) / len(values):.2f}" if values else "n/a"

    out = []
    for config in ["A", "B", "C"]:
        group = [r for r in rows if r["config"] == config]
        answered = [r for r in group if r["refused"] == "0"]
        out.append({
            "config": config,
            "accuracy": mean(r["correct_answer"] for r in group),
            "faithfulness": mean(r["grounded"] for r in answered) if config != "A"
else "n/a",
            "format_compliance": mean(r["format_ok"] for r in group),
            "robustness": mean(r["refused_when_needed"] for r in group),
            "retrieval_correct": mean(r["correct_retrieval"] for r in group),
        })
    write_csv(RAW / "rag_eval_summary.csv", out)
    print("| Config | Accuracy | Faithfulness | Format compliance | Robustness |
Retrieval correct |")
    print("|---|---|---|---|---|---|")

```

```

    for r in out:
        print(f"| {r['config']} | {r['accuracy']} | {r['faithfulness']} | {r['format_compliance']} | "
              f"{r['robustness']} | {r['retrieval_correct']} |")

def write_csv(path, rows):
    with path.open("w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--retrieve-only", action="store_true", help="print top-5 retrievals, no LLM calls")
    parser.add_argument("--summarize", action="store_true", help="summarize rag_eval.csv after manual grading")
    args = parser.parse_args()

    if args.summarize:
        summarize()
        return

    RAW.mkdir(parents=True, exist_ok=True)
    questions = yaml.safe_load(QUESTIONS.read_text(encoding="utf-8"))
    embed_model = HuggingFaceEmbedding(model_name=EMBED_MODEL)
    index, per_source = build_index(embed_model)
    print(f"chunks per source: {per_source} total: {sum(per_source.values())}")

    if args.retrieve_only:
        for q in questions:
            print(format_hits(q["id"], q["question"], 5, retrieve(index, q["question"], 5)))
        return

    client = OllamaModelClient(model=LLM_MODEL, num_predict=400)
    printouts, runs, evals = [], [], []
    for q in questions:
        hits = retrieve(index, q["question"], TOP_K)
        block = format_hits(q["id"], q["question"], TOP_K, hits)
        print(block)
        printouts.append(block)
        for config in ["A", "B", "C"]:
            rec = run_config(client, config, q, hits)
            runs.append(rec)

```

```

        evals.append(auto_eval(rec, q))
        print(f"  [{config}] {' '.join(rec['answer'].split())[:300]}")

by_id = {q["id"]: q for q in questions}
sweep = []
for qid in SWEEP_QUESTIONS:
    q = by_id[qid]
    for k in SWEEP_KS:
        hits = retrieve(index, q["question"], k)
        block = format_hits(qid, q["question"], k, hits)
        print(block)
        printouts.append(block)
        for config in ["B", "C"]:
            rec = run_config(client, config, q, hits)
            rec["k"] = k
            auto = auto_eval(rec, q)
            sweep.append({
                "id": qid,
                "config": config,
                "k": k,
                "retrieved": ";".join(h["chunk_id"] for h in hits),
                "irrelevant_retrieved": sum(h["source"] not in
q["expected_sources"] for h in hits),
                "kept": sum(c["kept"] for c in rec["chunks"]),
                "correct_retrieval": auto["correct_retrieval"],
                "input_tokens": rec["input_tokens"],
                "latency_ms": rec["latency_ms"],
                "correct_answer": "",
                "answer": auto["answer"],
            })
            print(f"  [{config}] k={k} {auto['answer'][:300]}")

(RAW / "rag_retrievals.txt").write_text("\n\n".join(printouts) + "\n")
(RAW / "rag_runs.json").write_text(json.dumps(runs, indent=2))
write_csv(RAW / "rag_eval.csv", evals)
write_csv(RAW / "rag_k_sweep.csv", sweep)
print(f"\nwrote rag_retrievals.txt, rag_runs.json, rag_eval.csv, rag_k_sweep.csv
to {RAW}")

if __name__ == "__main__":
    main()

```

Output:

```
{ date: python rag.py; } 2x1 | tee -a ../reports/hw04/RUN_LOG.txt
Sun Sep 27 22:34:00 PDT 2026
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|#####| 103/103 [00:00<00:00, 6914.20it/s]
chunks per source: { Password_Storage_Cheat_Sheet.md: 11, 'Session_Management_Cheat_Sheet.md': 25, 'cors.md': 3, 'oauth2-jwt.md': 6, 'sql-databases.md': 9} to
== Q1 | k=3 | What is the default max_age for FastAPI's CORSMiddleware, and what does it control?
#1 score=0.5224 cors.md#2 the allowed origins. ## Use 'CORSMiddleware' { #use-corsmiddleware } You can configure it in your **FastAPI** application using the
#2 score=0.4585 cors.md#3 'False'. None of 'allow_origins', 'allow_methods' and 'allow_headers' can be set to '['*']' if 'allow_credentials' is set to 'True'
#3 score=0.2496 sql-databases.md#5 (Press CTRL+C to quit) ``` </div> Then go to the '/docs' UI, you will see that **FastAPI** is using these **models** to **
[A] The default 'max_age' for FastAPI's 'CORSMiddleware' is **None**. ## What does 'max_age' control? The 'max_age' parameter in the 'CORSMiddleware' contr
sponse ('OPTIONS' request). This is part of the **CORS (C
[B] The default 'max_age' for FastAPI's 'CORSMiddleware' is **600 seconds** (or 10 minutes). ## What does 'max_age' control? The 'max_age' parameter controls
request). - When a browser makes a preflight request, the
[C] The default max_age for FastAPI's CORSMiddleware is 600 seconds. It controls the maximum time in seconds for browsers to cache CORS responses [2].
== Q2 | k=3 | What cookie attributes should a session ID cookie use, and what idle timeout does OWASP recommend?
#1 score=0.6799 Session_Management_Cheat_Sheet.md#17 shorter the session interval is, the lesser the time an attacker has to use the valid session ID. The se
#2 score=0.6578 Session_Management_Cheat_Sheet.md#22 the server-side idle timeout functionality with client-side code is that the user can see that the sessi
#3 score=0.6306 Session_Management_Cheat_Sheet.md#11 disk by the web browser based until the expiration time. Typically, session management capabilities to t
[A] When setting up a **session ID cookie** for web applications, it's important to follow best practices to ensure **security and proper session management**
recommended by **OWASP** (Open Web Application Security Pro
[B] **Answer:** A session ID cookie should use the following **cookie attributes** to ensure security and prevent session hijacking: - **HttpOnly**: Prevents
cross-site scripting (XSS) attacks. - **Secure**: Ensures
[C] A session ID cookie should use the 'HttpOnly', 'Secure', and 'SameSite=Strict' attributes to prevent accidental transmission over the wire in a non-secure
le to JavaScript executing in the origin [3]. OWASP recomm
== Q3 | k=3 | Which algorithm is recommended for hashing user passwords?
#1 score=0.5651 Password_Storage_Cheat_Sheet.md#1 # Password Storage Cheat Sheet ## Introduction This cheat sheet advises you on the proper methods for stori
#2 score=0.5469 Password_Storage_Cheat_Sheet.md#8 that use less secure hashing algorithms, such as MD5 or SHA-1, can be upgraded to modern password hashing
#3 score=0.5098 Password_Storage_Cheat_Sheet.md#3 the number of permutations can be enormous, with high speed hardware (such as GPUs) and cloud services with
[A] For hashing user passwords, the **recommended algorithm** is **bcrypt**, followed by **Argon2** and **scrypt** as strong alternatives. Here's a breakdown
**Designed specifically for password hashing**. - **Slower
[B] **Answer:** The recommended algorithm for hashing user passwords is **Argon2id** with a minimum configuration of **19 MiB of memory**, an **iteration coun
a minimum CPU/memory cost parameter of **2<sup>27</sup>**, a mini
[C] The recommended algorithm for hashing user passwords is **Argon2id** with a minimum configuration of 19 MiB of memory, an iteration count of 2, and 1 degr
== Q4 | k=3 | What is the recommended timeout?
#1 score=0.6096 Session_Management_Cheat_Sheet.md#17 shorter the session interval is, the lesser the time an attacker has to use the valid session ID. The se
```

Tables:

Six questions					
Question	Type	Top-1 score	A	B	C
Q1	answer in one chunk	0.52	wrong (None)	correct	correct
Q2	answer needs two chunks	0.68	wrong (30 min)	correct, adds unsupported attributes	correct
Q3	similar info across documents	0.57	wrong (bcrypt first)	correct	correct
Q4	ambiguous	0.61	generic, not from docs	correct, invents "every 30 minutes"	correct, says there is no single value
Q5	not in documents	0.47	hallucinated (4 hours)	answered "2 weeks" from outside knowledge	refused
Q6	unrelated	0.23	declined	declined	refused (no LLM call)

Source: raw/rag_retrievals.txt (printed top-k before each LLM call), raw/rag_runs.json , raw/rag_eval.csv

Evaluation summary

Config	Accuracy	Faithfulness	Format compliance	Robustness	Retrieval correct
A	0.17	n/a	n/a	0.83	n/a
B	0.83	0.40	0.00	0.83	0.50
C	1.00	1.00	1.00	1.00	0.50

- Accuracy: correct answer over the 6 questions (for Q5/Q6, correct = refused).
- Faithfulness: share of non-refused answers fully supported by the chunks in the prompt.
- Format compliance: answer cites [n] or is exactly the refusal sentence.
- Robustness: refused Q5/Q6 and did not refuse Q1–Q4.
- Retrieval correct: all expected sources were in the chunks given to the model; for Q5/Q6 in config C, no chunk survived the filter.

Refusal was detected automatically with a regex, then checked by hand. Q6 A ("I cannot provide personal opinions") and Q6 B ("subjective... ask your instructor") were missed by the regex and corrected to refused.

Source: `raw/rag_eval_summary.csv`

Top-k sweep

Question	k	Config	Chunks kept	Input tokens	Latency (ms)	Correct
Q2	1	B	1	538	9626.7	yes (cookie part from outside knowledge)
Q2	1	C	1	666	3161.4	no (cookie part wrong)
Q2	3	B	3	1518	15515.4	yes
Q2	3	C	3	1700	7352.0	yes
Q2	5	B	5	2495	14630.4	yes
Q2	5	C	5	2731	8154.6	yes
Q3	1	B	1	535	3202.6	yes
Q3	1	C	1	662	2640.7	yes
Q3	3	B	3	1513	7823.0	yes
Q3	3	C	3	1693	5686.7	yes
Q3	5	B	5	2491	12300.6	yes
Q3	5	C	5	2723	12697.1	yes

Source: `raw/rag_k_sweep.csv`

Analysis:

Retrieval worked for the questions whose wording matched one document. Q1 pulled two cors.md chunks, including the max_age line. Q2 got three chunks from the OWASP session cheat sheet: the timeout chunk ranked first, and the HttpOnly; Secure; SameSite=Strict sentence was in the third. Retrieval was only partly right for Q3 and Q4. For Q3, the Password Storage cheat sheet is about hashing from start to finish, so it filled every slot even at $k = 5$, and the short Argon2 section in FastAPI's oauth2-jwt.md never appeared. The answer was still correct, but the cross-document part was lost. For Q4, all three chunks were about session timeouts, so the CORS reading of "timeout" never reached the model.

The biggest improvement came from the grounding rules. With the same three chunks, B added facts from the model's own training: cookie Domain advice in Q2, a made-up "renew every 30 minutes" in Q4, and Django's real "2 weeks" default in Q5. Its accuracy was 0.83, but only 2 of its 5 non-refused answers were fully supported by the context. C used the same chunks, labelled them with source and chunk id, and told the model to cite and refuse. It answered all six questions correctly, with every claim cited, and each [n] could be traced back to a chunk_id.

The score filter helped on two questions. It removed an unrelated sql-databases.md chunk from Q1, so C's prompt was 1,021 input tokens against B's 1,388, even with the rules added. It dropped all three chunks for Q6 (top score 0.23), so C refused without calling the LLM. De-duplication never triggered: with 50-token overlap, neighbouring chunks share too few words to reach the 0.8 threshold. Chunk size stayed at 500 in all runs, so its effect was not tested.

On Q5 and Q6, the model hallucinated when it had no rules. Without RAG, it said Django sessions last 4 hours (wrong). B said 2 weeks, which is correct but not in the documents. On Q6, both A and B declined only because the question is subjective, not because of the missing evidence. C refused both questions with the exact required sentence. Q5 shows the limits of similarity scores: its chunks scored 0.44–0.47, close to Q1's 0.52, so the filter could not tell "on topic" from "contains the answer". The refusal came from the prompt, not from retrieval.

The k sweep shows more context is not always better. Q2 needed $k = 3$. At $k = 1$, C answered the cookie part from a passage about expiring cookies at logout. At $k = 5$, input tokens rose by about 60% with no gain. Q3 was already correct at $k = 1$, and $k = 5$ only made the answer longer and about 2x slower. Overall, retrieval decides what the model can know, the context decides what it sees, and the rules keep it inside that context.